CIS437 - Cloud Computing

– Cloud Design Patterns I –

Foundations, Pub/Sub, and Geode

Instructor: Yong Zhuang

yong.zhuang@gvsu.edu

Based on the original version by Professor Erik Fredericks

Last Class

- Why Load Balancing?
- Load Balancer Types: Application vs. Network
- Frontends, Backends, and Health Checks
- Load Balancing with MIGs and Autoscaling
- External vs. Internal Load Balancing
- Hands-on Lab
- Practice

Last Class Practice

Scenario

A company runs a public web application on a managed instance group. Traffic varies during the day, and one VM may occasionally fail.

1. Which type of load balancer is appropriate for public HTTP/HTTPS traffic?
2. What determines whether a VM should receive new requests?
3. What adds or removes VM instances as demand changes?
4. If the application should only be reachable from inside the VPC, which type of load balancer should we use?



Agenda

- What Is a Design Pattern?
- Why Cloud Applications Need Design Patterns
- Cloud Fallacies
- Publish/Subscribe Pattern
- Geode Pattern
- Compare the Patterns
- Practice

What Is a Design Pattern?

A design pattern “…names, abstracts, and identifies the key aspects of a common design structure that make it useful for creating a reusable object-oriented design.”*

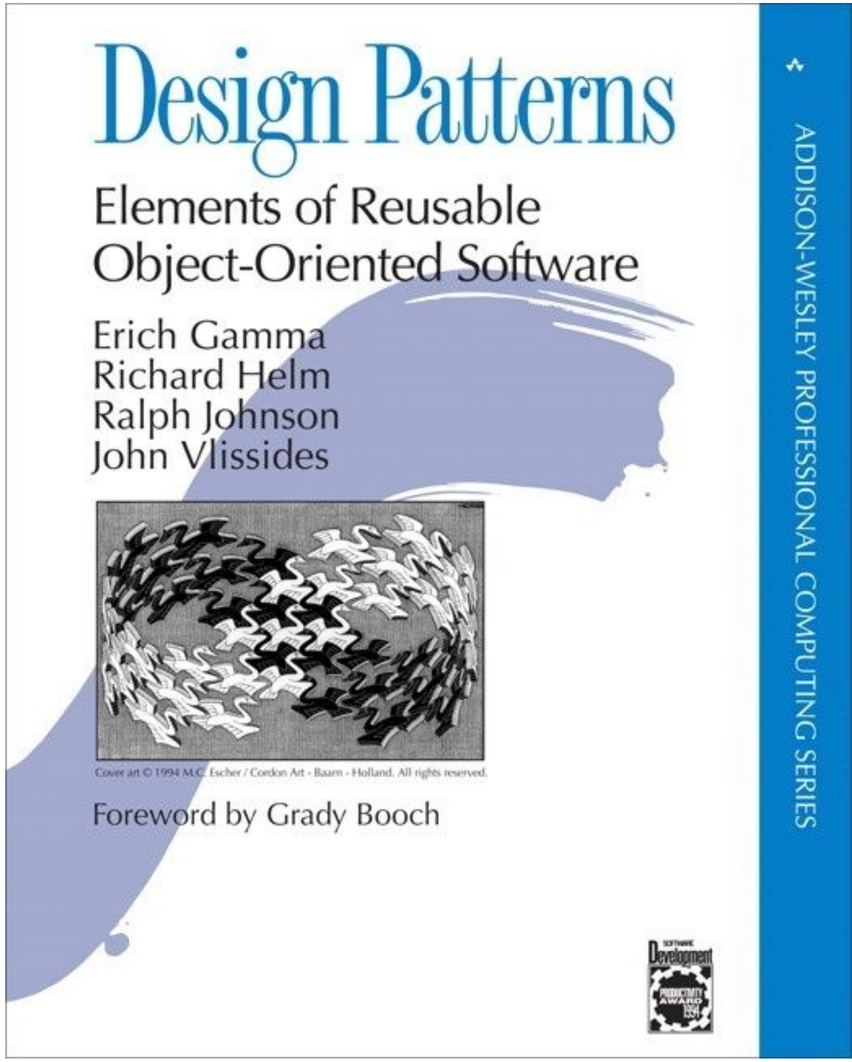
A design pattern is a design pattern is a proven solution to a recurring problem in a particular context .

A reusable way to structure and communicate a solution to a common software-design problem.



What do we mean by proven?

How does communication fit in?



*From the book pictured

Why Study Design Patterns?

- Reuse proven solutions to commonly recurring problems
- Establish a common vocabulary for discussing system design
- Think about architecture at a higher level instead of individual implementation details
- Improve maintainability and modifiability
- Learn from solutions that have been tested in many systems

Design Patterns in Software Engineering

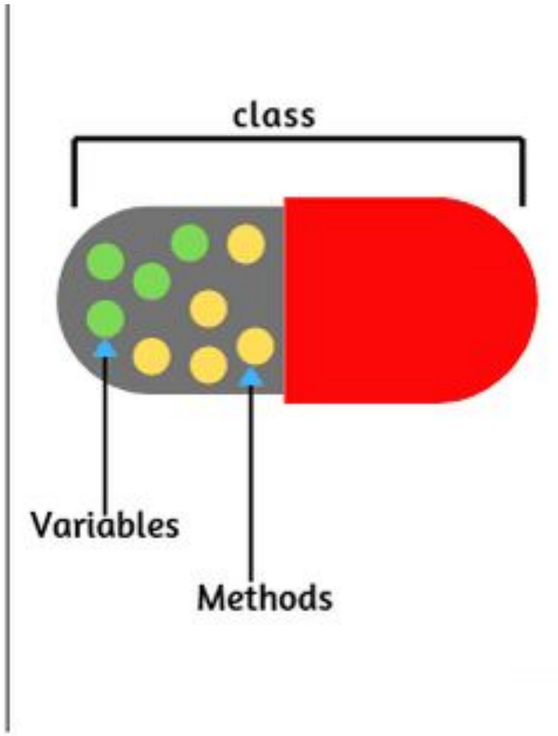
Adoption of improved object-oriented design strategies

- Encapsulation and information hiding
- Design to interfaces
- Favor composition over inheritance

```
class
{

  data members
        +
  methods (behavior)

}
```



Cloud Design Patterns

Cloud design patterns apply the same design-pattern idea to recurring problems in cloud applications.

Common concerns include:

- Reliability and failure
- Scalability and changing load
- Latency
- Distributed communication
- Geographic distribution

Cloud design patterns = proven architectural solutions to recurring cloud problems

Cloud design patterns address recurring problems such as reliability, scalability, latency, distributed communication, and geographic distribution.

Why Cloud Applications Need Patterns

- Components and networks can fail
- Users may be geographically distributed
- Traffic can change unexpectedly
- Services may communicate across networks
- Latency and bandwidth are not unlimited
- Cloud components should be loosely coupled when possible

Cloud design must assume change and failure.

Awesome Software Architecture – Cloud Design Patterns

Cloud / Distributed Computing Fallacies

False assumptions we should not make when designing distributed systems.

- The network is reliable
- Latency is zero
- Bandwidth is infinite
- The network is secure
- Topology doesn't change
- There is one administrator
- Component versioning is simple
- Observability implementation can be delayed
  - i.e., monitoring and understanding what went wrong

A cloud application must be designed assuming that networks, components, and environments can change or fail.

Azure: Cloud design patterns

Fallacies of distributed computing

Publish/Subscribe Pattern

Asynchronous messaging that decouples message producers from consumers

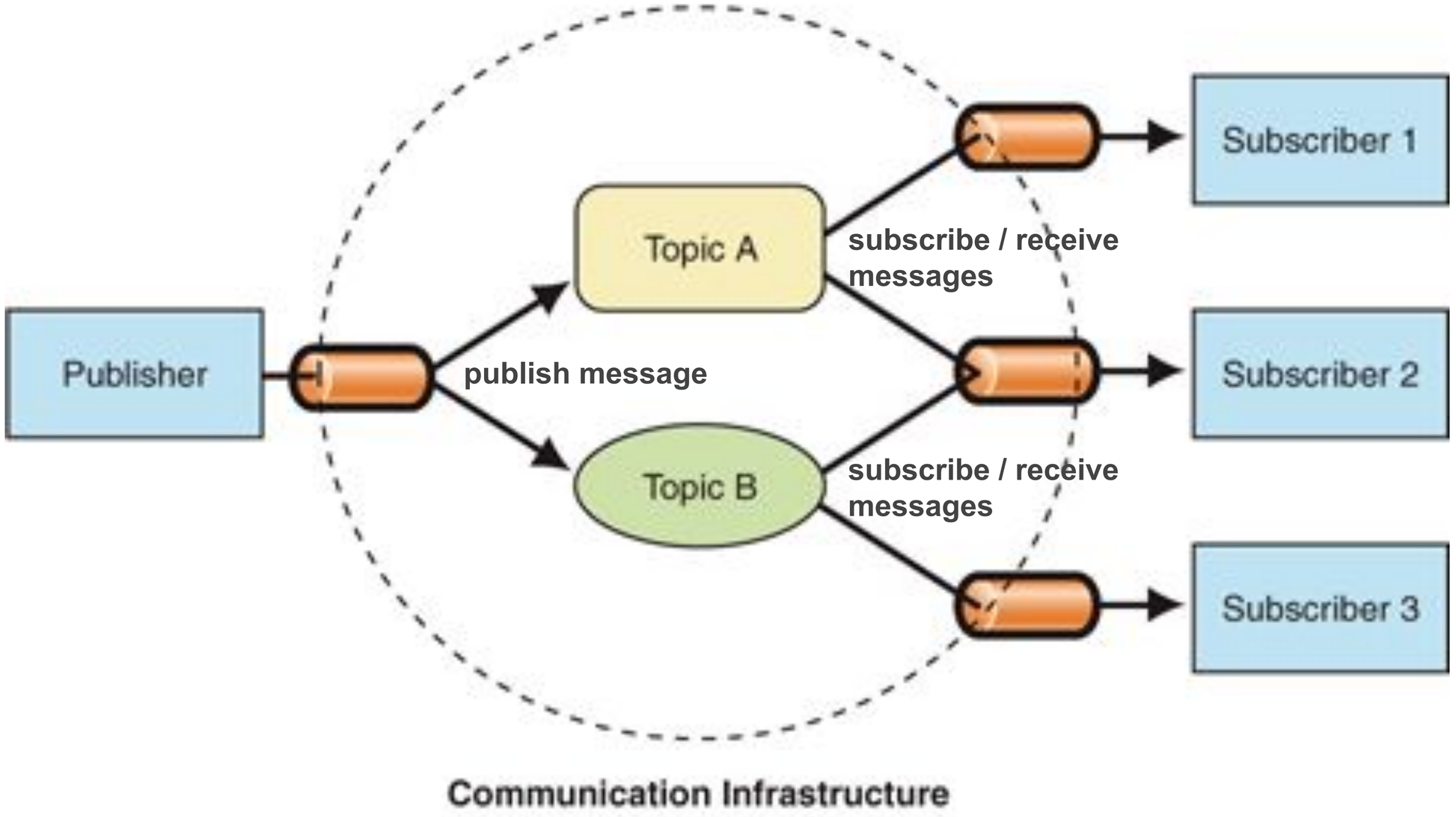
- **Publisher** — produces and sends messages to a topic
- **Topic / Broker** — receives and distributes messages
- **Subscriber** — receives messages from topics it subscribes to
- Publishers do **not need to know who the subscribers are**
- Common in **event-driven applications, IoT, and background processing**

"Publish/Subscribe is a design pattern. Google Cloud Pub/Sub is a Google Cloud service that implements this type of architecture."

[AWS: Publish-subscribe pattern]

[Azure: Publisher-Subscriber pattern]

Pub/Sub Diagram



Publisher, Subscriber, and Broker

Publisher

- Produces messages/events
- Publishes messages to a topic

Subscriber

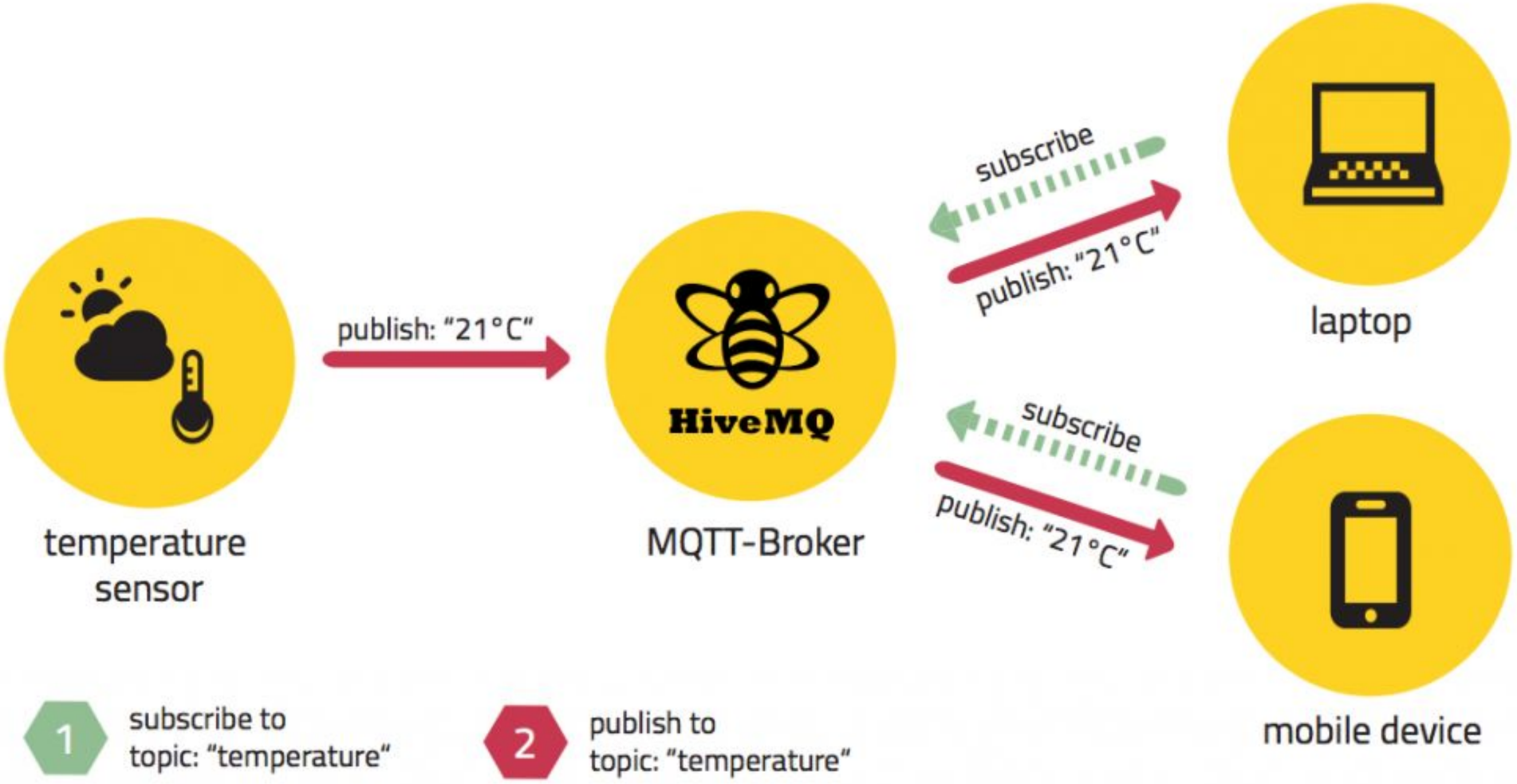
- Subscribes to one or more topics
- Receives messages from those topics

Broker / Messaging Service

- Receives published messages
- Tracks topics and subscriptions
- Delivers messages to subscribers
- May handle authentication, filtering, and delivery policies

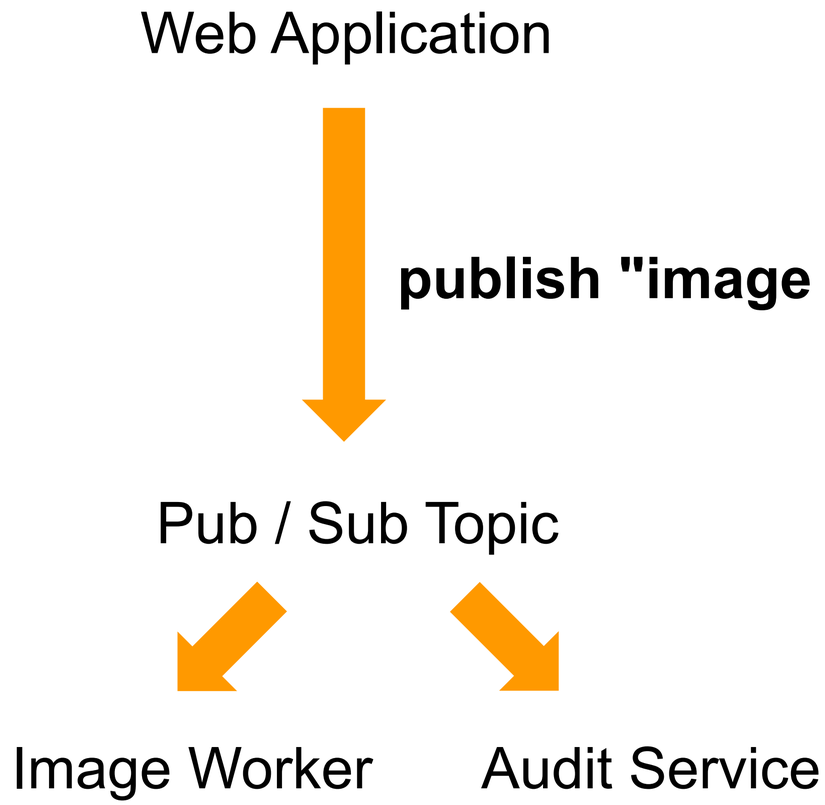
Publisher = produces
Subscriber = consumes
Broker = coordinates message delivery

IoT Pub/Sub Example



Google Cloud Pub/Sub Example

Pub/Sub enables asynchronous, decoupled communication.



The web application does not need to wait for image processing to finish.

Geode: A pattern for geographically distributed applications



Why Geode?

Goals

- Serve users in multiple geographic regions
- Improve availability
- Support large-scale global applications
- Reduce latency for distant users

Geode is motivated by global users, latency, scale, and availability—but multi-region deployment also creates new traffic, deployment, and data challenges.

Challenges

- Network latency and traffic management
- Deploying the application consistently across regions
- Keeping geographically distributed data available and consistent

How Could We Solve It?

Scenario:

Our application has users in North America, Europe, and Asia, but it currently runs in only one region.

How could we redesign the application to:

- reduce latency for distant users?
- improve availability if one geographic location fails?

Geode Pattern

Geode Pattern

- Deploy the application in multiple geographic regions
- Each geographic deployment is a geode
- Each geode should be able to serve requests independently

Use automation to keep deployments consistent

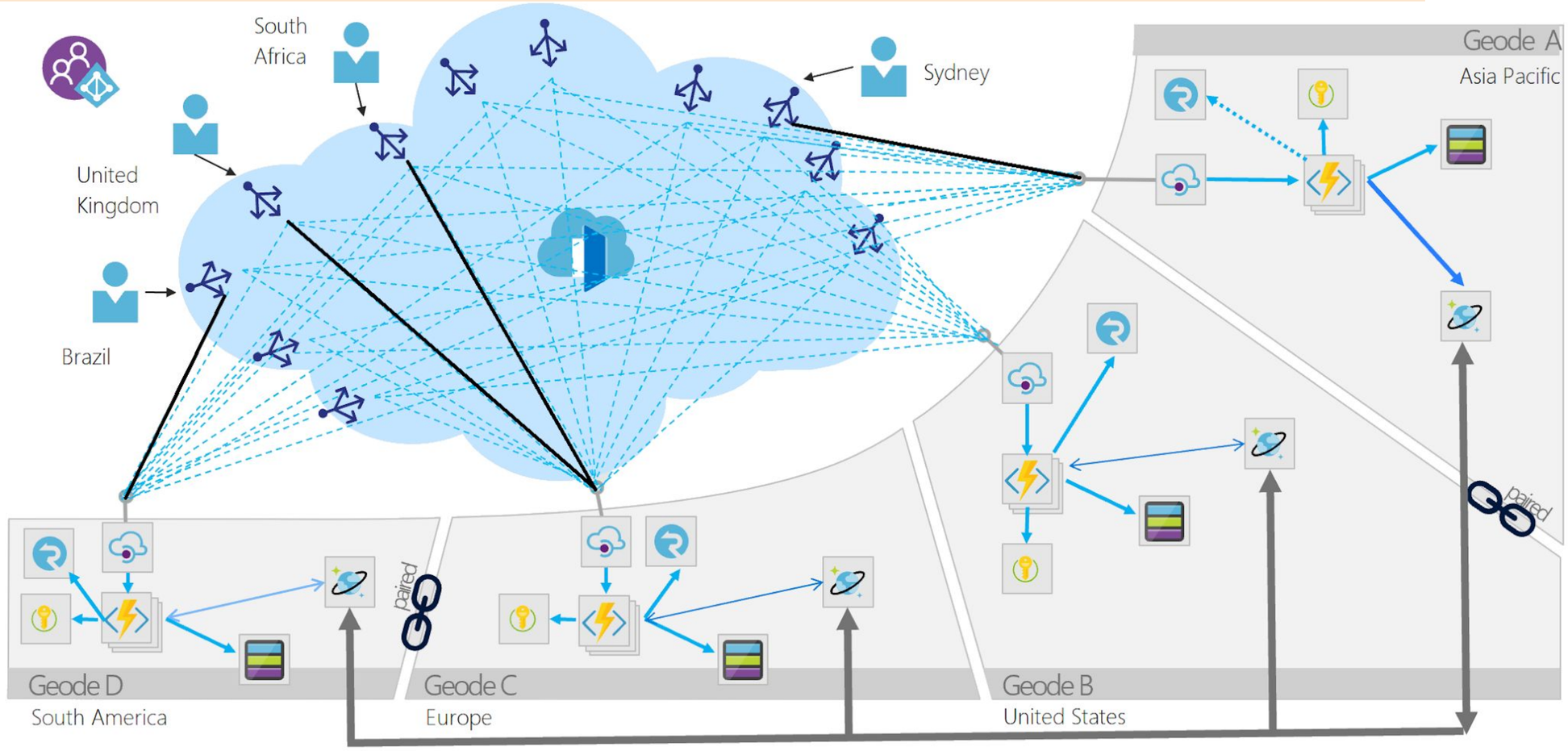
- Deploy from templates / infrastructure definitions
- Use CI/CD to update multiple regions
- Use load balancing or traffic management to direct users to an appropriate healthy geode

A Geode architecture requires two or more geodes

Geode = multiple self-contained geographic deployments

Geode Architecture Diagram

Each geode contains compute + local/replicated data
Users are directed to an appropriate healthy geode



[Azure: Geode pattern]

Compare Pub/Sub and Geode

	Publish/Subscribe	Geode
Problem	Producers and consumers are tightly coupled	Users are geographically distributed; one deployment creates latency/availability risks
Key idea	Asynchronous messaging through topics	Multiple geographic deployments
Main benefit	Decoupling	Lower latency + geographic availability
Example	IoT sensors → subscribers	Global web application
Focus	How components communicate	Where the application is deployed

Summary

- Design patterns are proven solutions to recurring problems.
- Cloud applications must be designed for **change, latency, and failure**.
- **Publish/Subscribe** uses asynchronous messaging to decouple producers and consumers.
- **Geode** uses multiple geographic deployments to improve latency and availability.
- **Pub/Sub focuses on communication; Geode focuses on geographic deployment.**
- Choose a pattern based on the **problem you are trying to solve**.

Resources / Further Reading

- Awesome Software Architecture – Cloud Design Patterns
- Microsoft Azure Architecture Center – Cloud Design Patterns
- AWS Cloud Design Patterns
- TechTarget – Cloud Design Patterns
- DoiT – Cloud Landing Zone Anti-patterns
- Google Cloud Pub/Sub Overview

Practice

Scenario

1. A factory has hundreds of sensors continuously producing temperature and equipment-status events. Several applications need to consume those events independently. **Which pattern fits best? Why?**
2. An online service has customers in North America, Europe, and Asia. It needs low latency and should remain available even if one geographic deployment fails. **Which pattern fits best? Why?**
3. A global e-commerce company runs its application in several geographic regions. When a customer places an order, multiple independent services need to process the event—for example, inventory, shipping, and analytics. **Which pattern or patterns could be useful? Why?**

